

# Parlar Ioniano

## Paper work pack

Present your own language: hand-written HTML, CSS and JavaScript, Bootstrap and data-driven grammar tables

**Weeks 36–49 · no project work in weeks 41–43 · handover Fri 4 Dec 2026**

This pack is a schedule and a checklist for the moments when the site is not open. The project journal is written on the site and committed to the repository's project-docs folder. A tick in this booklet is not a submission — the work always lives in the Git repository.

The repository is public: no personal data, no school identifiers, no other people's names or faces.

The language material is the author's own. The site shows the credit and the terms of use.

## Schedule

| Wk | Dates          | Week's topic                               | Phase |
|----|----------------|--------------------------------------------|-------|
| 36 | 31 Aug – 4 Sep | Kickoff, toolchain and the first deploy    | A     |
| 37 | 7–11 Sep       | Content becomes data: the Alphabet view    | A     |
| 38 | 14–18 Sep      | Bootstrap shell and the component decision | A     |
| 39 | 21–25 Sep      | Pronouns: one function, eight tables       | B     |
| 40 | 28 Sep – 2 Oct | Nouns, articles and gender — and v0.5      | B     |
| 44 | 26–30 Oct      | Client review and re-plan                  | B     |
| 45 | 2–6 Nov        | The number converter — built from scratch  | C     |
| 46 | 9–13 Nov       | Phrasebook, word cards and search          | C     |
| 47 | 16–20 Nov      | Quality and testing week                   | C     |
| 48 | 23–27 Nov      | Release candidate, user test and v1.0      | D     |
| 49 | 30 Nov – 4 Dec | Demonstration week                         | D     |

Hand in by Fri 4 Dec 2026. The site has the detailed instructions, the implementation help and the project journal.

# Phase A — Foundations: brief, toolchain, plan and a published shell

## Week 36 · 31 Aug – 4 Sep — Kickoff, toolchain and the first deploy

*After this week an empty but real site — plain `index.html` with Bootstrap loaded from a CDN — is live at your public GitHub Pages URL, the plan is drafted, and the P0 work is split into issues.*

- ☐ Hold the kickoff talk: bring your question list and record every answer as a decision, an open item or an assumption.
- ☐ Set up VS Code, the site folder served over `http://localhost` and the public repository — privacy check first, 2FA on.
- ☐ Create a hotspot on your phone, connect your computer to it, open the local server on the phone, and publish the empty site to GitHub Pages over your own connection.
- ☐ Draft the technical plan below and split the P0 pages into prioritised issues.

**Done when:** The public URL opens on a device that has never seen the project, and another person understands from the README and the plan what is being built and for whom.

**Work sample to the Git repository before ticking:** Kickoff notes with the question list, the repository with its README, the first deploy live at the public URL, the plan draft, the P0 backlog — and the phone screenshot in `project-docs/evidence/week-36/`.

## Week 37 · 7–11 Sep — Content becomes data: the Alphabet view

*The first real content is live: the Alphabet & Pronunciation page renders every letter, its IPA value and the digraphs — all fetched from a JSON file, none of it typed into the HTML.*

- ☐ Type `data/letters.json` and the first records of `data/words.json` from your PDF (pages 7, 9–11, 12 and 41) and document the data-store decision in the plan.
- ☐ Build `js/data.js` as the shared loader and the Alphabet & Pronunciation page on a runtime fetch with loading and error states.
- ☐ Run `tools/check-data.html`, add one check of your own and prove it fails on a planted error.

**Done when:** The public URL shows the alphabet table rendered from `data/letters.json`, and `tools/check-data.html` reports "content OK" — and reports an error when you deliberately break one line.

**Work sample to the Git repository before ticking:** `data/letters.json` and `data/words.json`, `js/data.js`, the Alphabet page live at the public URL, and the `tools/check-data.html` report pass/fail in the journal.

## Week 38 · 14–18 Sep — Bootstrap shell and the component decision

*The site has a real shell: a Bootstrap navbar and grid on every page, a Home page that explains the language — and Bootstrap taken into use from a CDN, configured in the language's colours, measured, and its limits written down.*

- ☐ Write the comparison memo: Bootstrap, one other component library and the no-library option, with your own transferred-KB and Lighthouse measurements.
- ☐ Load Bootstrap from the CDN with a pinned version, configure the theme in `css/style.css`, and build the navbar, the grid and the Home page.
- ☐ Measure again from the DevTools Network panel and record the before/after numbers in the memo.

**Done when:** The comparison memo with your own measured numbers is in `project-docs/`, the navbar works from every page on the public URL and collapses correctly on a phone, and the Home page shows the credit and terms.

**Work sample to the Git repository before ticking:** The comparison memo with your own measured numbers is in `project-docs/`, the navbar works from every page on the public URL and collapses correctly on a phone, and the Home page shows the credit and terms.

# Phase B — Grammar views: data-driven tables and the client review

## Week 39 · 21–25 Sep — Pronouns: one function, eight tables

*The Pronouns page is live: eight pronoun tables (subject, object, possessive, reflexive, imperative, direct/indirect, prepositional, double object) in Bootstrap tabs, all built by ONE reusable render function from data/grammar.json.*

- ☐ Design the pronouns section of data/grammar.json so one shape serves all eight tables, and type the data from PDF pages 13–18 and 22–26.
- ☐ Write the reusable renderTable() in js/render.js and wire the eight tables into Bootstrap tabs.
- ☐ Make the loading and error states real and capture both as screenshots.

**Done when:** All eight pronoun tables render on the public URL from data/grammar.json, the tabs can be operated with the keyboard alone, wide tables scroll horizontally on a phone instead of breaking the layout, and renaming the JSON shows your error state.

**Work sample to the Git repository before ticking:** All eight pronoun tables render on the public URL from data/grammar.json, the tabs can be operated with the keyboard alone, wide tables scroll horizontally on a phone, and renaming the JSON shows your error state.

## Week 40 · 28 Sep – 2 Oct — Nouns, articles and gender — and v0.5

*The Nouns, Articles & Gender page is live: gender rules, plural endings, definite/indefinite articles and the articulated prepositions — each rule section in a Bootstrap accordion, each matrix built by the same renderTable(). Before the break: v0.5 is tagged and the week-44 review is booked, in writing.*

- ☐ Add the nouns and articles sections to data/grammar.json from PDF pages 8 and 19–21, including the elision rules.
- ☐ Render the gender-by-number matrices with the reused renderTable() inside a Bootstrap accordion, and add the example sentences.
- ☐ Tag v0.5 and deploy it.
- ☐ Book the week-44 review in writing: names, date and tasks into project-docs/review-agreement.md.

**Done when:** The four grammar pages render correctly on the public URL, the accordion opens and closes with the keyboard alone, v0.5 is tagged, and the review agreement (names + date) is committed in project-docs/.

**Work sample to the Git repository before ticking:** The four grammar pages render correctly on the public URL, the accordion opens and closes with the keyboard alone, v0.5 is tagged, and the review agreement (names + date) is committed in project-docs/.

## Week 44 · 26–30 Oct — Client review and re-plan

*The client and an external tester have used v0.5 at the public URL; their findings are recorded in their own words; the backlog is re-prioritised; the first complete debugging chain is documented from a real review finding.*

- ☐ Run the review session and record the tester's findings verbatim, separated from your interpretation.
- ☐ Turn the client's decisions into issues and re-prioritise the backlog, comparing estimates with reality.
- ☐ Fix a small batch and document debugging chain 1/3 end to end.
- ☐ Check the t12 open item (device-security policy) with the supervisor and update the plan.

**Done when:** The review log is committed with the two voices clearly separated, every finding has a decision (fix / won't fix / P2), and chain 1/3 is complete from a real finding, with its regression test in the matrix.

**Work sample to the Git repository before ticking:** project-docs/review-2026-w44.md with the two voices separated, the updated backlog, the fix commits, and chain 1 in debugging-chains.md.

## Phase C — Finish: feedback change, quality and testing

### Week 45 · 2–6 Nov — The number converter — built from scratch

*The Numbers & Comparison page is live with a number converter built from scratch: type 319, get "Trecentodecenove" — pure functions in js/numbers.js, unit-tested in the browser against the PDF before the code ran.*

- ☐ Write the converter tests first: expected values from PDF pages 42–45, committed while still red.
- ☐ Implement numberTolonian and the ordinal helpers in js/numbers.js as small pure functions until tests/tests.html reports 0 failed.
- ☐ Build the converter UI from scratch — no Bootstrap components — and ship the week-44 feedback change.

**Done when:** On the public URL, at least 0–999 999 converts correctly including the separation and clipping rules, tests/tests.html reports 0 failed with at least 6 converter tests whose names cite PDF pages, and the feedback change is live.

**Work sample to the Git repository before ticking:** tests/numbers.test.js with PDF-cited expectations, js/numbers.js, the Numbers page live, and the feedback-change commit linked to its finding.

### Week 46 · 9–13 Nov — Phrasebook, word cards and search

*The Phrasebook & Chapter 1 page is live: the café dialogues side by side with an Ionian/English toggle, the vocabulary as a Bootstrap card grid, and a search box that filters the cards as you type.*

- ☐ Complete data/words.json and write data/phrases.json from PDF pages 12 and 51–56, marking pages 57–58 as “coming later”.
- ☐ Render the vocabulary as a Bootstrap card grid and build the dialogue viewer with the Ionian/English toggle.
- ☐ Build the search that filters the cards in memory, render with textContent, and prove the XSS string appears as text.
- ☐ Fill the test matrix to at least 12 cases and commit LICENSE and CREDITS per the client's decision.

**Done when:** A beginner can find 'thank you' by typing 'thank' on the public URL, the cards filter as you type with a live match count, the dialogues toggle between languages, searching for the T10 attack string (the img-onerror line in the test matrix) renders as harmless text, and the full test matrix (at least 12 cases) exists with the expected results filled in.

**Work sample to the Git repository before ticking:** The Phrasebook live with the card grid and search, the XSS screenshot, the full test matrix with expected results filled in, and the LICENSE/CREDITS commits.

### Week 47 · 16–20 Nov — Quality and testing week

*The site passes a real quality bar: the full test matrix is run with results recorded, all three debugging chains are complete, accessibility is verified with before/after Lighthouse pairs, every page validates as HTML, and the code has been refactored while every test stays green.*

- ☐ Execute the whole test matrix against the public URL and fill the observed column with dates.
- ☐ Complete debugging chain 3/3 and compile chains 1–3 into one document.
- ☐ Run the accessibility audit and the W3C HTML validation: fix keyboard, contrast, alt, labels and markup errors, and record the Lighthouse before/after pairs.
- ☐ Review the CDN dependencies (version pinned, integrity hash) and run the no-secrets check, then refactor in small commits with the tests green in between.

**Done when:** The matrix has every row filled with observed results and dates, three complete chains are in debugging-chains.md, the after-Lighthouse accessibility score is recorded next to its before-pair, every page validates without errors, and tests/tests.html plus tools/check-data.html are both green on main.

**Work sample to the Git repository before ticking:** The executed test matrix, debugging-chains.md with three complete chains, the Lighthouse pairs in project-docs/lighthouse/, html-validation.md and security-review.md, and the refactor commit series.

## Phase D — Release and demonstration

### Week 48 · 23–27 Nov — Release candidate, user test and v1.0

*Version 1.0 is released: content frozen, tested in a clean environment, verified by an external user who learned a café dialogue from the site alone — and the repository documentation lets a stranger run the project without asking you anything.*

- ☐ Freeze the content and tag the release candidate.
- ☐ Run the clean-environment test from a fresh clone using only the README, fixing the README where it fails.
- ☐ Run the external user test with a written task card, recording the tester's words verbatim and separately.
- ☐ Write the deployment-path description from your own artefacts and release v1.0.

**Done when:** The tester reached and read the café dialogue using only the site and your written instructions — no spoken help; the fresh clone runs with the README steps only; v1.0 is tagged, released and live at the public URL.

**Work sample to the Git repository before ticking:** The clean-clone notes, user-test-2026-w48.md, deployment-path.md, the instruction-fix commits and the v1.0 release.

### Week 49 · 30 Nov – 4 Dec — Demonstration week

*Nothing new is added. Every one of the 44 requirements points to its exact work sample, the 8–10 minute demo is rehearsed, and the project is handed over.*

- ☐ Link all 44 matrix rows below to their exact work samples.
- ☐ Commit the final project journal and AI log, and write the self-assessment.
- ☐ Rehearse the 8–10 minute demo twice with a timer and hand over on Friday.

**Done when:** Every matrix row has a working link, the journal covers all 11 weeks, the demo ran inside 10 minutes in rehearsal, and the handover package is complete on Friday.

**Work sample to the Git repository before ticking:** The fully linked matrix, project-docs with the final journal, AI log and self-assessment, and the handover confirmation.

## The last five days

**Mon 30 Nov** · Content freeze — the last accepted version

**Tue 1 Dec** · Evidence: journal, tests and matrix links

**Wed 2 Dec** · Rehearse the 8–10 min demo and the self-assessment

**Thu 3 Dec** · Buffer — final check with another person

**Fri 4 Dec** · Demonstration and handover

# Evidence matrix — 44 competence requirements

Tick only when the requirement has an exact work sample: a link, a commit, a screenshot, a test row or a memo. The same work sample can serve several requirements. Requirement names are shown in Finnish exactly as in the national qualification criteria (ePerusteet).

## Tieto- ja viestintätekniikan perustehtävät

- ☐ **Palveluasenne** – How you receive and act on feedback in the review and the user test (wk 44, 48).
- ☐ **Asiakkaiden palvelu** – Kickoff and review with the client role: the question list and the recorded answers (wk 36, 44).
- ☐ **Tarpeen varmistaminen** – The needs analysis and the justified solution proposal in the plan (wk 36).
- ☐ **Palautteen pyytäminen** – The feedback request in the review; the self-assessment in the demonstration week (wk 44, 49).
- ☐ **Viestintäkanavat** – GitHub issues and the agreed channels in use for the whole project (wk 36 onwards).
- ☐ **Alan perussanasto** – The comparison memo: field vocabulary and the library options with their trends (wk 38).
- ☐ **Englanninkielinen materiaali** – English documentation sources cited in the journal and the AI log (wk 37–39).
- ☐ **Tiedonhaku ongelmassa** – The information trail inside the debugging chains (wk 44–47).
- ☐ **Käyttöjärjestelmät** – The development environment set up: OS tools, VS Code, a local web server, Git (wk 36).
- ☐ **Verkkoyhteyden jako** – The local web server shared over your own phone hotspot and tested on the phone, with the security notes (wk 36).
- ☐ **Verkon perusrakenne** – The deployment-path description from your own artefacts: localhost → LAN → Pages/HTTPS (wk 48).
- ☐ **Laitteiden suojaus** – Partial: environment hardening (wk 36), the checkpoint (wk 44) and the dependency audit (wk 47); the rest is an open item owned by the supervisor.

## Ohjelmointi

- ☐ **Kehitysympäristö** – VS Code, Live Server and browser DevTools in use; the first commit and deploy (wk 36).
- ☐ **Virheiden korjaus** – Three complete debugging chains, observation to regression test (wk 44–47).
- ☐ **Toimintojen testaus** – The converter unit tests in tests/tests.html and the executed test matrix (wk 45, 47).
- ☐ **Rakenteinen ohjelmointi** – The converter's pure functions and ES-module split in js/numbers.js (wk 45).
- ☐ **Ylläpidettävä koodi** – The refactor series with green tests, naming and HTML validation (wk 47).
- ☐ **Käyttöliittymän toteutus** – The pages built from the site map: Home, Alphabet, Pronouns, Nouns & Articles (wk 38–40).
- ☐ **Toimintojen toteutus** – The converter and the search, from issue to done-when condition (wk 45–46).
- ☐ **Tehtävistä sopiminen** – The weekly agreements with the supervisor and the issue assignments (wk 36, 44).
- ☐ **Ratkaisujen etsiminen yhdessä** – The library options discussed together; the review solutions agreed together (wk 38, 44).
- ☐ **Toimivuuden arviointi** – The solutions evaluated in the review, with the recorded decisions (wk 44).
- ☐ **Oman toiminnan arviointi** – The self-assessment and the journal's weekly reflections (wk 49).

## Ohjelmistokehittäjänä toimiminen

- ☐ **Asiakkaan tarpeet** – The kickoff: needs recorded as decisions, open items and assumptions (wk 36).
- ☐ **Asiakaslähtöinen viestintä** – A technical matter explained in plain language in the review and test memos (wk 44, 48).
- ☐ **Version katselmointi** – The v0.5 review with a decision for every finding (wk 44).
- ☐ **Tärkeysjärjestys** – The P0/P1/P2 backlog with justifications (wk 36).
- ☐ **Tehtäviksi jako** – The views split into issues with done-when conditions (wk 36).
- ☐ **Toteutuksen suunnittelu ja arviointi** – The size estimates and the estimate-vs-actual comparison in the re-plan (wk 36, 44).
- ☐ **Toimintalogiikka** – The converter logic and the search (wk 45–46).
- ☐ **Tietovaraston valinta** – The JSON-files-in-data/ + runtime-fetch decision with the rejected alternatives (wk 37).
- ☐ **Yhteys tietovarastoon** – Fetch with loading and error states, implemented and tested (wk 37, 39).
- ☐ **Rajapinnat ja tiedon käsittely** – The Fetch API plus the data transforms: grouping, filtering, search (wk 39, 46).



- ☐ **Tietoturvan arviointi** – The textContent XSS proof, the CDN dependency review and the no-secrets check (wk 46–47).
- ☐ **Versionhallinta** – The commit history, the v0.5 and v1.0 tags, the branches and PRs (wk 36–49).
- ☐ **Osan liittäminen versioon** – The feedback change merged into the existing version from a feature branch (wk 45–46).
- ☐ **Julkaisu tuotantoon** – The first deploy from the main branch and the v1.0 release on GitHub Pages (wk 36, 48).

## Ohjelmiston toteuttaminen ohjelmistokomponenttikirjastolla

- ☐ **Kehittämisympäristön käyttöönotto** – The environment set up, and Bootstrap taken into use from the CDN and configured in css/style.css (wk 36, 38).
- ☐ **Mahdollisuudet ja rajoitteet** – The comparison memo naming Bootstrap's possibilities and limits with your own measurements (wk 38).
- ☐ **Toiminnot ja työkalut** – The navbar, grid, tables, tabs and accordion built on Bootstrap's components (wk 39–40).
- ☐ **Ulkoiset komponentit** – External components loaded from a CDN with pinned versions: Bootstrap CSS and JS, the icons, the fonts (wk 38).
- ☐ **Toteutus kirjastolla** – The pages designed, implemented and tested with Bootstrap (wk 39–46).
- ☐ **Julkaisu asiakkaalle** – Published to the client's environment: GitHub Pages, the public URL (wk 48).
- ☐ **Dokumentointi** – The README and the user documentation in the agreed form (wk 48).

*Remember: the ticks and text fields on the site are stored only in your browser. They do not reach the teacher and they never replace the work in Git.*